

Algorithms -

1. Sorting

- a. Selection
- b. Insertion
- c. Bubble
- d. Quick sort ✓
- e. Merge sort ✓
- f. Heap sort

2. Searching

- a. Linear
- b. Binary

10, 2, 3, 4, 5, 6, 7, 17 → 18

3. Greedy approach

- a. Dijkstra

Binary Search - sorted array, index random access

4. Divide and conquer

- a. Quick sort
- b. Merge sort

find 13 from the data set

1, 3, 4, 6, 7, 8, 10, 13, 15, 16, 18, 20

5. Dynamic programming

- a. 0/1 Knapsack

$\text{mid} = (\text{low} + \text{high}) / 2 + 1 = (0 + 12) / 2 + 1 = 7$

time complexity = $O(\log n)$

6. Hoffman coding

7. Misc

Analysis of Algorithms

Best case	The best case of an algorithm is the <u>minimum time</u> it takes to run for any input of size n. It happens when the input is in the most favorable condition.	In Linear Search, if the target element is found at the first position, the time complexity is - $O(1)$. So, this is the best case.
Worst case	The worst case of an algorithm is the <u>maximum time</u> it can take for any input of size n. It happens when the input is in the most difficult condition.	In Linear Search, if the target element is at the last position or not present, the algorithm checks all elements - $O(n)$. So, this is the worst case.
Average case	The average case is the expected running time for all possible inputs of size n. It represents the algorithm's typical performance.	In Linear Search, on average the element may be found in the middle of the list, so approximately $n/2$ comparisons are needed - $O(n)$

1	Big O	- Represents <u>upper bound</u> of a function - Show maximum growth rate of running time (worst case) $f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that $0 \leq f(n) \leq c * g(n)$ for all $n \geq n_0$
2	Omega (Ω)	- Represents <u>lower bound</u> of a function - Show minimum growth rate of running time (best case) $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and n_0 such that $0 \leq c * g(n) \leq f(n)$ for all $n \geq n_0$
3	Theta (Θ)	- Represents <u>tight bound</u> of a function - Show both minimum and maximum growth rate of running time (average case) $f(n) = \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$

1.

```
int num;
cin >> num;
for(int i = 1; i <= num; i+=2)
    if(num % i == 0)
        flag = false;
```

- $O(n)$
- $O(\log n)$
- $O(1)$

2.

```
for (in i)
    for (int j)
```

$O(n^2)$

Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

To check how these algorithms works visually, visit here - <https://see-algorithms.com/sorting/>

Insertion Sort	Selection Sort
[6, 3, 8, 2, 7, 1, 5, 4] <pass 1> [3, 6, 8, 2, 7, 1, 5, 4] <pass 2> [3, 6, 8, 2, 7, 1, 5, 4] <pass 3> [3, 6, 8, 8, 7, 1, 5, 4] <pass 3> [3, 6, 6, 8, 7, 1, 5, 4] <pass 3> [3, 3, 6, 8, 7, 1, 5, 4] <pass 3> [2, 3, 6, 8, 7, 1, 5, 4] <pass 3> [2, 3, 6, 8, 7, 1, 5, 4] <pass 4> [2, 3, 6, 7, 8, 1, 5, 4] <pass 5> [2, 3, 6, 7, 8, 8, 5, 4] <pass 5> [2, 3, 6, 7, 7, 8, 5, 4] <pass 5> [2, 3, 6, 6, 7, 8, 5, 4] <pass 5> [2, 3, 3, 6, 7, 8, 5, 4] <pass 5> [2, 2, 3, 6, 7, 8, 5, 4] <pass 5> [1, 2, 3, 6, 7, 8, 5, 4] <pass 5> [1, 2, 3, 6, 7, 8, 5, 4] <pass 6> [1, 2, 3, 6, 7, 8, 8, 4] <pass 6> [1, 2, 3, 6, 7, 7, 8, 4] <pass 6> [1, 2, 3, 6, 6, 7, 8, 4] <pass 6> [1, 2, 3, 5, 6, 7, 8, 4] <pass 7> [1, 2, 3, 5, 6, 7, 8, 8] <pass 7>	[6, 3, 8, 2, 7, 1, 5, 4] 6 3 8 2 7 1 5 4 1 3 8 2 7 6 5 4 1 2 8 3 7 6 5 4 1 2 8 3 7 6 5 4 1 2 3 8 7 6 5 4 1 2 3 8 7 6 5 4 1 2 3 4 7 6 5 8 1 2 3 4 7 6 5 8 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 min = 1 min = 2 min = 2 min = 3 min = 3 min = 4 min = 4 min = 5 min = 5 min = 5 min = min = 5 min = 5

[1, 2, 3, 5, 6, 7, 7, 8] <pass 7>	
[1, 2, 3, 5, 6, 6, 7, 8] <pass 7>	
[1, 2, 3, 5, 5, 6, 7, 8] <pass 7>	
[1, 2, 3, 4, 5, 6, 7, 8] <pass 7>	

Quick Sort - worst case $O(n^2)$
average case $O(n \log n)$

pivot = array/dataset er last element

```

10 4 9 3 5 2 6
4 10 9 3 5 2 6
4 3 9 10 5 2 6
4 3 5 10 9 2 6
4 3 5 2 9 10 6
4 3 5 2 6 10 9

```

```

4 3 5 2      6      10 9
4 3 5 2      6      9 10
2 3 5 4      6      9 10

```

```

2      3 4 5 6 9 10

```

Merge Sort - average and worst case $O(n \log n)$

```

10 4 9 3 5 2 6
10 4 9 3      5 2 6
10 4      9 3      5 2 6
10 4      9 3      5 2 6

4 10      3 9      2 5      6
3 4 9 10      2 5 6
2 3 4 5 6 9 10

```

Heap Sort -
increasing order = min
decreasing order = max

```

10 4 9 3 5 2 6
10

```

// 2, 3, 4, 5, 6, 9, 10

Dynamic Programming -

Discuss 0/1 Knapsack with an example

Exercise 01 - A file contains the following characters with the frequencies as shown.

Character	Frequency
a	10
e	15
i	12
o	3
u	4
s	13
t	1

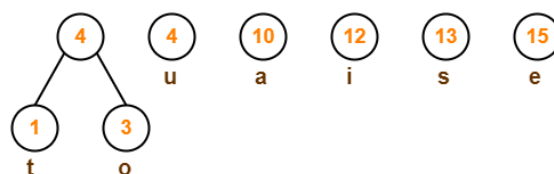
If Huffman Coding is used for data compression, determine-

1. Huffman Code for each character
2. Average code length
3. Length of Huffman encoded message (in bits)

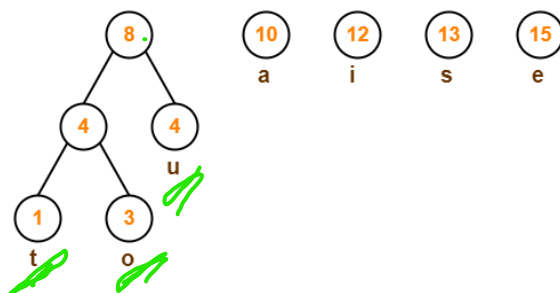
Step-01:



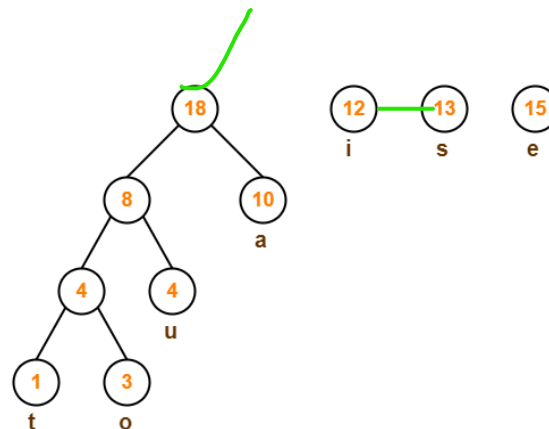
Step-02:



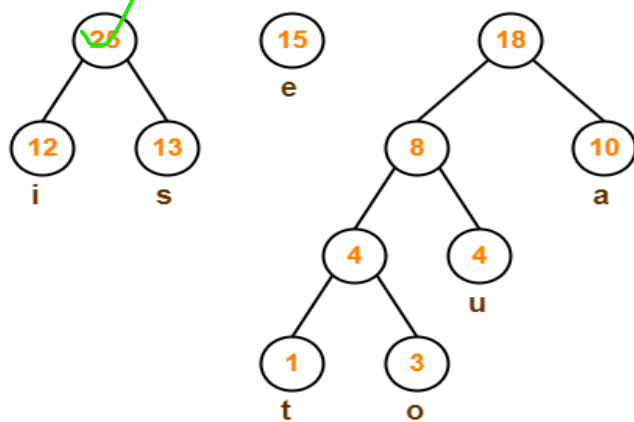
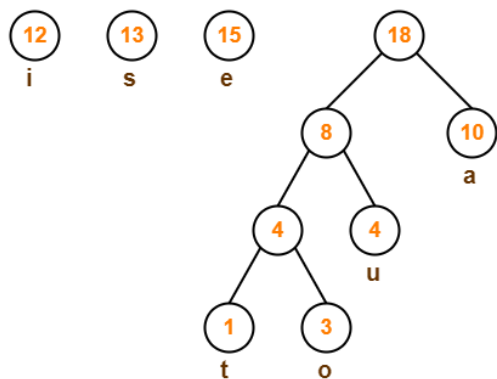
Step-03:



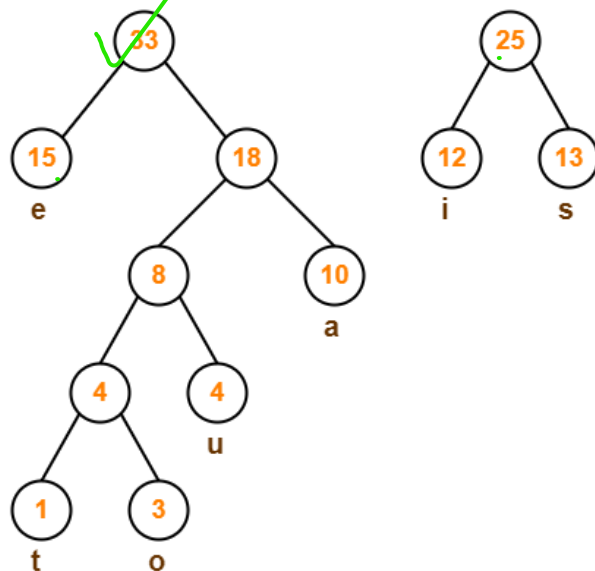
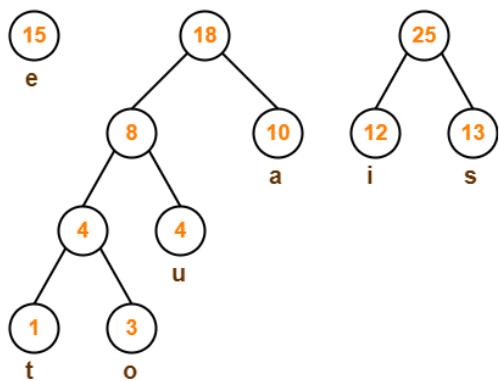
Step-04:



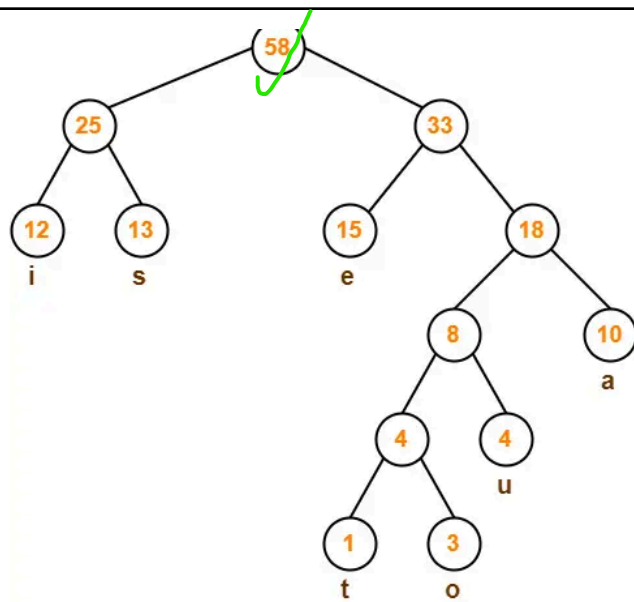
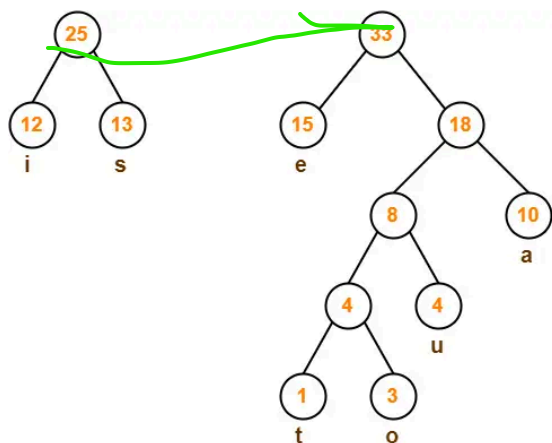
Step-05:

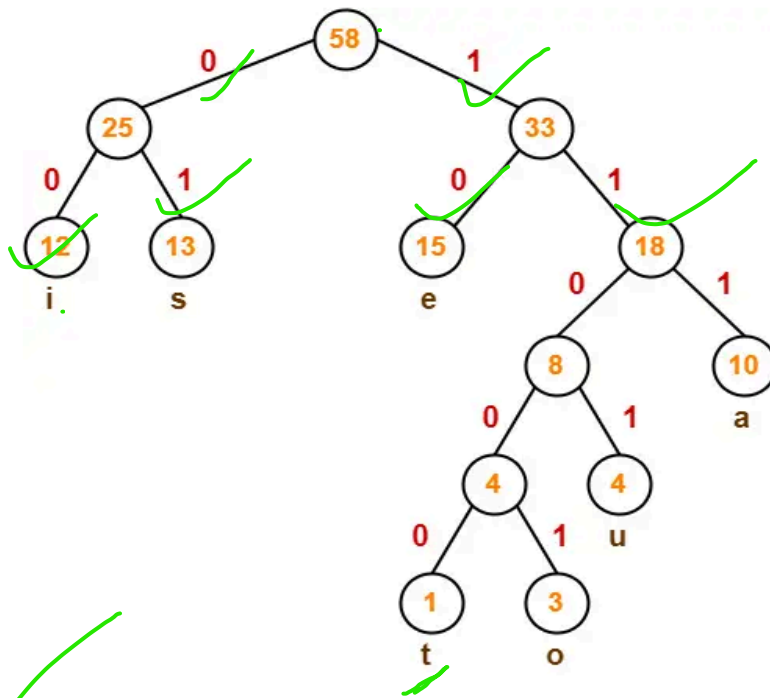


Step-06:



Step-07:





- a = 111
- e = 10
- i = 00
- o = 11001
- u = 1101
- s = 01
- t = 11000

Average code length = $\sum (\text{frequency}_i \times \text{code length}_i) / \sum (\text{frequency}_i)$
 $= \{ (10 \times 3) + (15 \times 2) + (12 \times 2) + (3 \times 5) + (4 \times 4) + (13 \times 2) + (1 \times 5) \} / (10 + 15 + 12 + 3 + 4 + 13 + 1) = 2.52$

Total number of bits in Huffman encoded message
 $= \text{Total number of characters in the message} \times \text{Average code length per character}$
 $= 58 \times 2.52 = 146.16 \approx 147 \text{ bits}$